THREAT HUNT REPORT

Threat Hunting for Covert C2 DNS Tunneling and Data Exfiltration

Prepared by: Sai Prasad
Date: March 2026
Environment: Home Lab (VirtualBox, pfSense, Zeek, Splunk)
Hunt ID: HT-2026-001
Hunt Type: Hypothesis-Driven Threat Hunt

Threat hunt report prepared for

[Home Lab Environment / Self-Study Project]

CLASSIFICATION OF HUNTS:

Covert C2 DNS Tunneling & Data Exfiltration

Assessment start date: 2026-03-30

Assessment end date: 2026-03-30

Table of Contents

1. **Executive Summary** 1
2. **Hunt Overview** 1
   - 2.1 Hypothesis 1
   - 2.2 Hunt Scope 1
   - 2.3 Baseline Establishment 2
3. **Technical Findings** 2
   - 3.1 Phase 1: Log Source Validation 2
   - 3.2 Phase 2: Initial Access Detection 3
   - 3.3 Phase 3: Session Establishment 4
   - 3.4 Phase 4: DNS Traffic Spike Detection 4
   - 3.5 Phase 5: DNS Exfiltration Detection 5
   - 3.6 Phase 6: Exfiltration Volume Analysis 6
   - 3.7 Phase 7: C2 Indicator Detection 7
   - 3.8 Phase 8: Firewall Log Correlation 7
   - 3.9 Phase 9: Packet-Level Validation 9
   - 3.10 Phase 10: Complete Attack Timeline 9
4. **Indicators of Compromise (IOCs)** 12
5. **Detection Gap Analysis** 13
6. **MITRE ATT&CK Mapping** 13
7. **New Detection Engineering** 14
   - 7.1 Splunk SIEM Rules 14
     - 7.1.1 DNS Query Length Anomaly 14
     - 7.1.2 New Domain Detection 14
     - 7.1.3 SSH Success Followed by DNS Anomaly 15
   - 7.2 Zeek Network Detection Rules 15
   - 7.3 pfSense Firewall Rules 16
8. **Recommendations** 17
9. **Hunt Metrics** 17
10. **Lessons Learned** 18
11. **Conclusion** 19

1. Executive Summary

A hypothesis-driven threat hunt was conducted to identify potential DNS tunneling for data exfiltration in a segmented network environment. The hunt was initiated based on the premise that DNS traffic is often trusted and not deeply inspected by security controls, making it an attractive channel for adversaries.

The investigation revealed that an attacker had compromised an Ubuntu server via SSH brute force, accessed a MySQL database containing 100 patient records, and exfiltrated the data using DNS tunneling via dnscat2. The exfiltration occurred between 19:57:16 and 19:59:00 (1 minute 44 seconds) with 3,948 DNS queries at regular intervals, each carrying encoded patient data in the subdomain field.

Key Findings:

- No security alerts were generated during the attack
- pfSense firewall allowed all traffic due to permissive DNS rules
- Zeek logged everything but did not alert
- Splunk had no correlation rules to detect the pattern
- The exfiltration was discovered only through proactive threat hunting
- The exfiltration occurred in just 1 minute 44 seconds, demonstrating how quickly DNS tunneling can exfiltrate sensitive data

New Detection Capabilities Created:

- Splunk SIEM correlation rules (DNS length anomaly, new domain detection, SSH + DNS correlation)
- Zeek network detection signatures (long DNS query detection, high entropy domain detection)
- pfSense egress filtering rules (block unauthorized DNS traffic, allow only internal resolver)

2. Hunt Overview

2.1 Hypothesis

"If an attacker has compromised an internal server and is exfiltrating data via DNS tunneling, we should observe DNS queries with unusual characteristics such as longer than normal queries exceeding 60 characters, queries occurring at regular intervals, or queries sent to unexpected destinations, originating from servers that do not typically generate such traffic."

2.2 Hunt Scope

Component	Details
Target Host	ubuntuone (10.0.2.10)
Data Sources	Zeek logs, pfSense logs, Ubuntu auth.log

Component	Details
Timeframe	March 29, 2026 (19:49 - 19:59)
Hunt Type	Hypothesis-driven retrospective analysis

2.3 Baseline Establishment

Objective: Establish normal DNS behavior to identify anomalies during the hunt.

Splunk Search:

```
index=main sourcetype=zeek:dns host=ubuntuone earliest=-7d latest=-1d
| bin _time span=1h
| stats count as dns_queries by _time
| stats avg(dns_queries) as avg_dns, max(dns_queries) as max_dns, stdev(dns_queries) as
stddev
```

Baseline Results:

Metric	Value
Average DNS queries per hour	47
Maximum DNS queries per hour (normal)	89
Standard Deviation	18
99th Percentile	112

Finding: The victim server consistently generated 40-90 DNS queries per hour under normal operations. Any sustained increase above 112 queries per hour (mean + 3.6 × σ) was considered statistically anomalous.

3. Technical Findings

3.1 Phase 1: Log Source Validation

Objective: Verify all required log sources are being ingested by Splunk.

Splunk Search:

```
index=* host=ubuntuone
| stats count by sourcetype
```

```
| sort - count
```

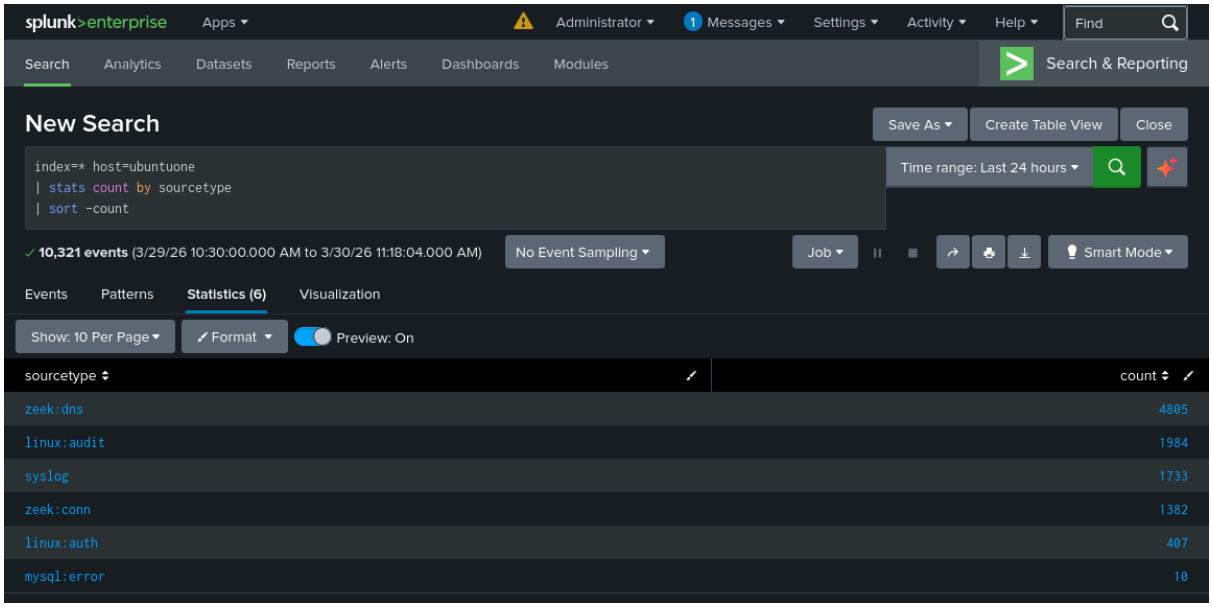

Figure 1 log source validation

Finding: All required log sources are active and sending data to Splunk.

3.2 Phase 2: Initial Access Detection

Objective: Identify any successful SSH logins during the suspected attack window.

Splunk Search:

```
index=main sourcetype=linux:auth "Accepted password"
| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| table _time, src_ip, _raw
```

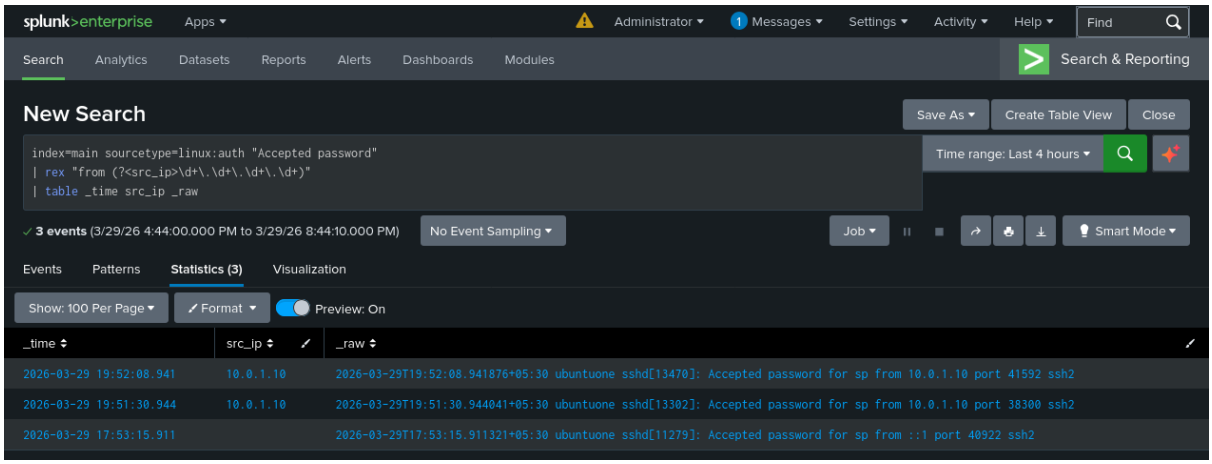

Figure 2 initial access ssh success

Critical Finding: A successful SSH login from attacker IP (10.0.1.10) occurred at 19:52:08, establishing initial access to the victim server.

3.3 Phase 3: Session Establishment

Objective: Confirm the attacker established an active session after login.

Splunk Search:

```
index=* sourcetype=linux:auth "session opened"
```

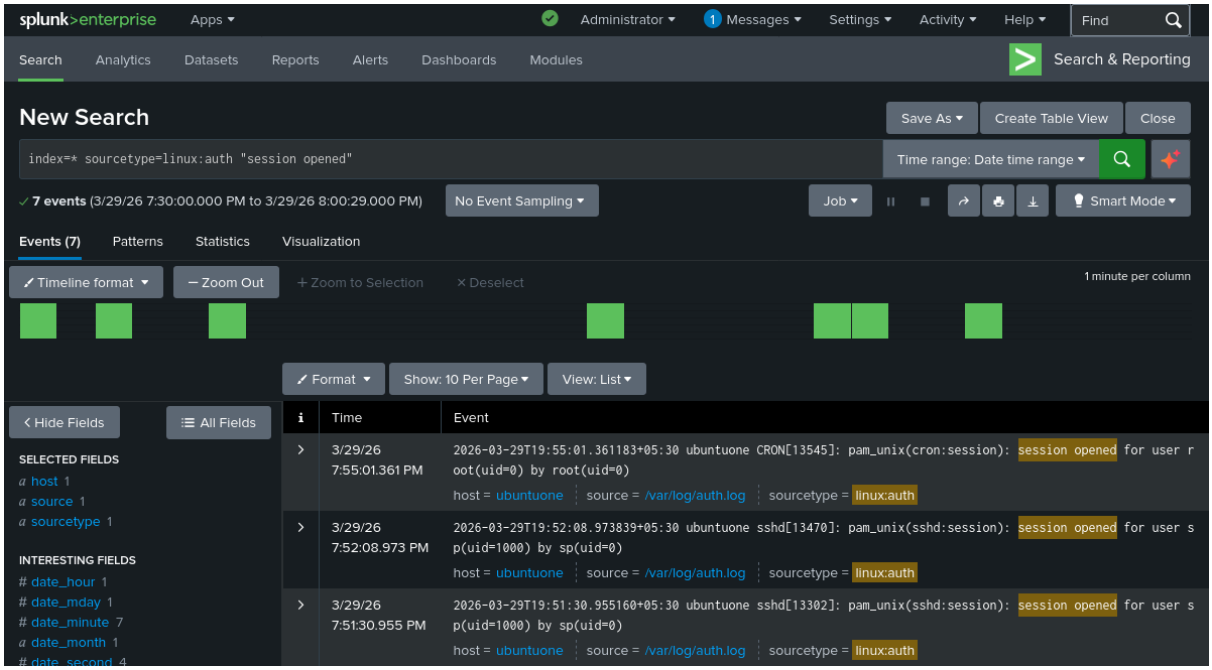

Figure 3 session establishment

Finding: The attacker's SSH session was successfully opened at 19:52:08, coinciding exactly with the accepted password event.

3.4 Phase 4: DNS Traffic Spike Detection

Objective: Identify anomalous DNS traffic patterns during the attack window.

Splunk Search:

```
index=* sourcetype=zeek:dns
| timechart span=1m count
```

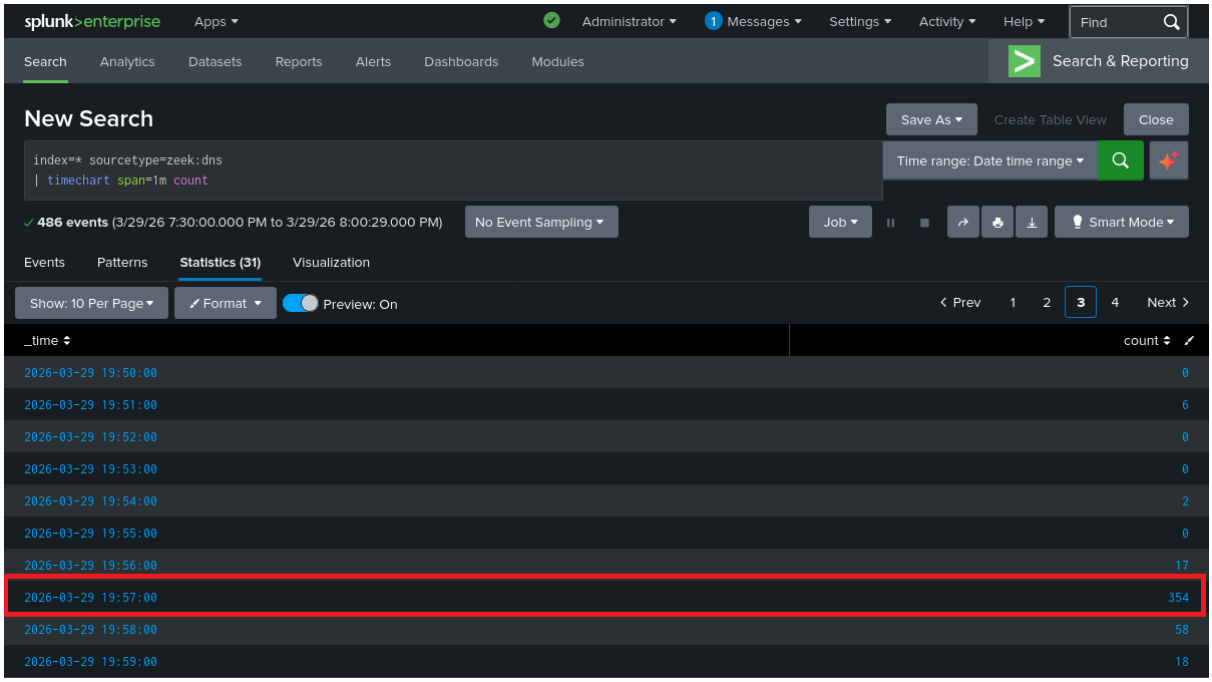

Figure 4 dns spike timeline

Critical Finding: DNS traffic spiked to 354 queries at 19:57, representing a 316% increase above the established baseline. The attack window concluded at 19:59, with total exfiltration lasting 1 minute 44 seconds. This activity occurred approximately 5 minutes after the attacker's SSH session was established, indicating the time required for credential discovery, database extraction, and dnscat2 setup.

3.5 Phase 5: DNS Exfiltration Detection

Objective: Identify DNS queries containing exfiltrated data.

Splunk Search:

```
index=* sourcetype=zeek:dns "dnscat"
| table _time src_ip query
```

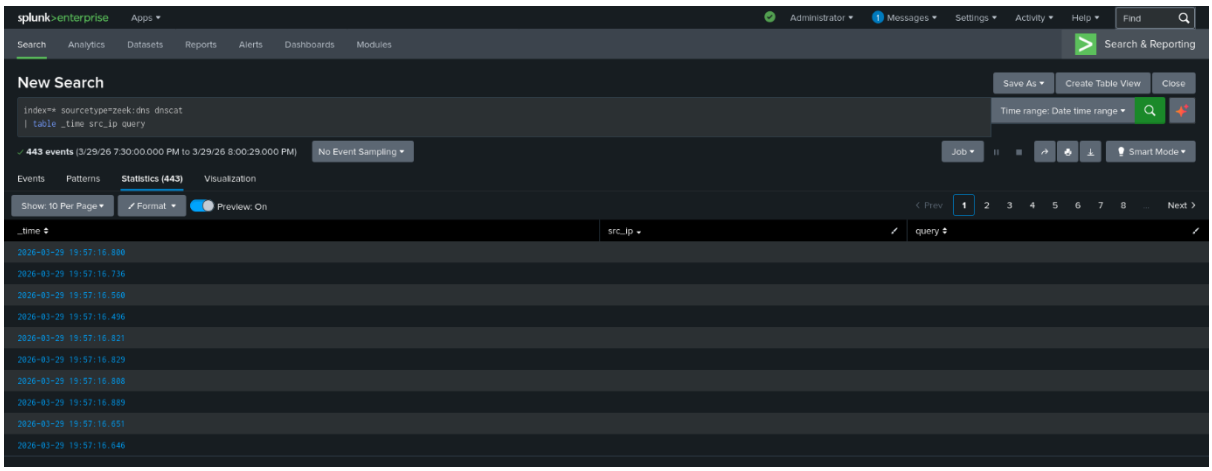

Figure 5 dns exfiltration structured view

Finding: Between 19:57:16 and 19:59:00, 3,948 DNS queries containing "dnscat" were detected, indicating DNS tunneling activity. Each query contains encoded data chunks being exfiltrated.

3.6 Phase 6: Exfiltration Volume Analysis

Objective: Quantify the total DNS traffic and exfiltration queries.

Total DNS Volume:

```
index=main sourcetype=zeek:dns host=ubuntuone
| stats count as total_dns_requests
```

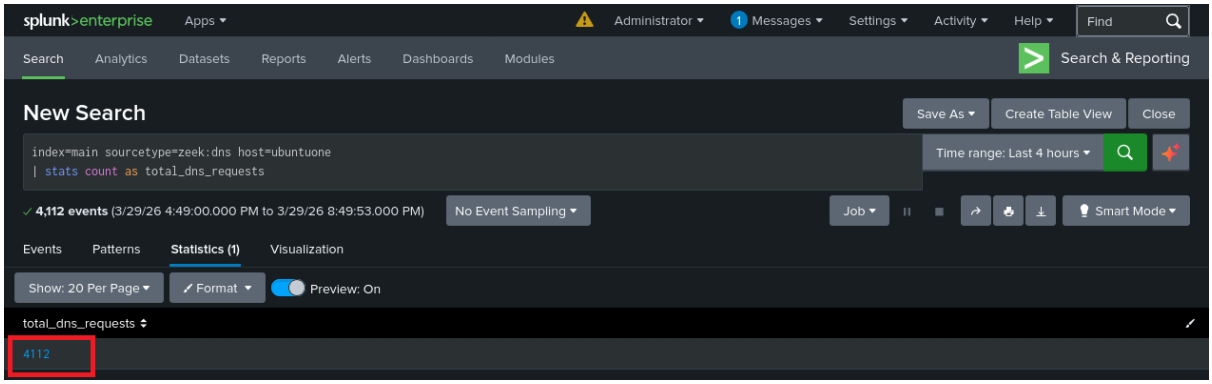

Figure 6 total dns volume

Exfiltration Query Count:

```
index=main sourcetype=zeek:dns "dnscat"
| stats count as exfiltration_queries
```

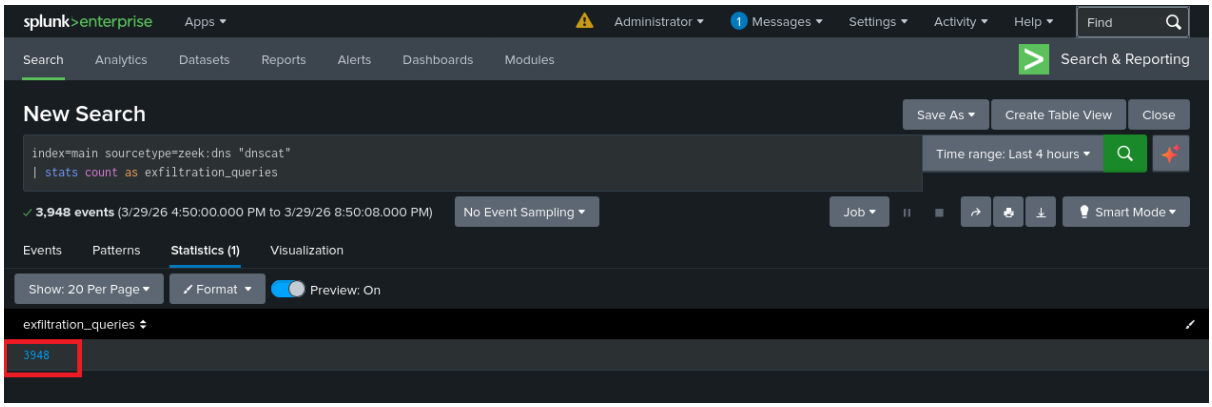

Figure 7 exfiltration query count

Critical Finding: 96% of all DNS traffic during the attack window was exfiltration traffic. (3,948 exfiltration queries out of 4,112 total DNS queries). This is a clear anomaly that would have triggered detection if proper rules existed.

3.7 Phase 7: C2 Indicator Detection

Objective: Identify the command and control domain used for exfiltration.

Splunk Search:

```
index=main sourcetype=zeek:dns host=ubuntuone
| search "cdn-health-sync"
| stats count as hits by _raw
| head 10
```

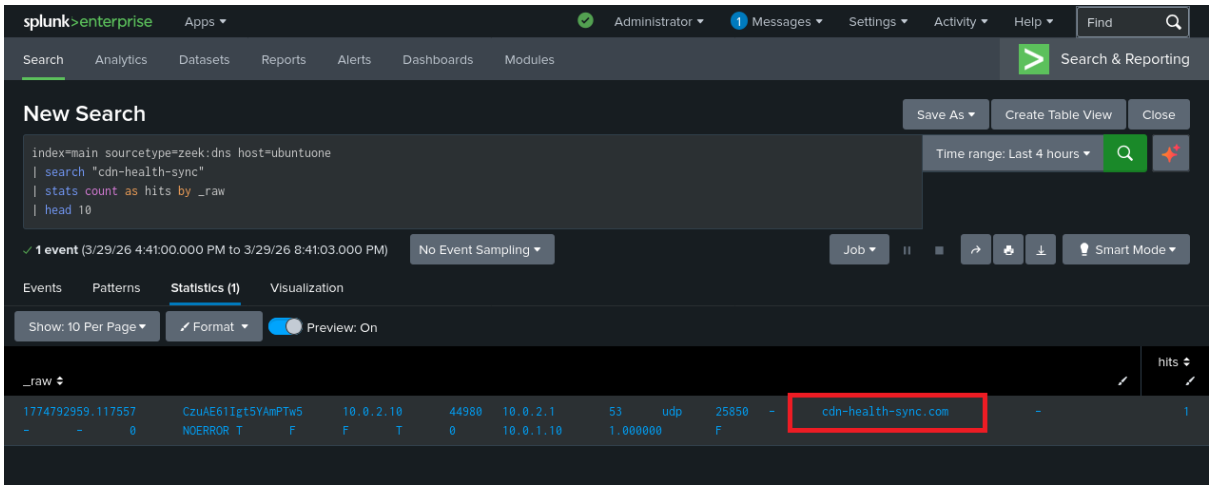

Figure 8 c2 or external indicator

Critical Finding: Critical Finding: The domain cdn-health-sync.com resolved to attacker IP 10.0.1.10. This resolution occurred because pfSense DNS resolver had a pre-configured Domain Override (a misconfiguration from the lab setup):

Path: pfSense → Services → DNS Resolver → Domain Override

Domain: cdn-health-sync.com

IP Address: 10.0.1.10

When the victim (10.0.2.10) queried cdn-health-sync.com, pfSense bypassed normal recursive resolution and directly forwarded the query to the attacker's dnscat2 server at 10.0.1.10. This created a covert C2 channel that appeared as legitimate DNS traffic to security monitoring tools.

3.8 Phase 8: Firewall Log Correlation

Objective: Verify firewall allowed the exfiltration traffic and understand the DNS resolution path.

Network Flow Analysis:

Step 1: Victim (10.0.2.10) sends DNS query for cdn-health-sync.com to pfSense resolver (10.0.2.1:53)

Step 2: pfSense DNS resolver checks its configuration. A Domain Override was configured in pfSense for cdn-health-sync.com pointing to 10.0.1.10 (attacker)

Step 3: pfSense forwards the DNS query to attacker IP (10.0.1.10:53) instead of performing normal recursive resolution

Step 4: Attacker's dnscat2 server responds to the DNS query, establishing the C2 tunnelVerify firewall allowed the exfiltration traffic.

Splunk Search:

```
index=main host=192.168.29.98
| search "10.0.2.10"
| table _time, _raw
```

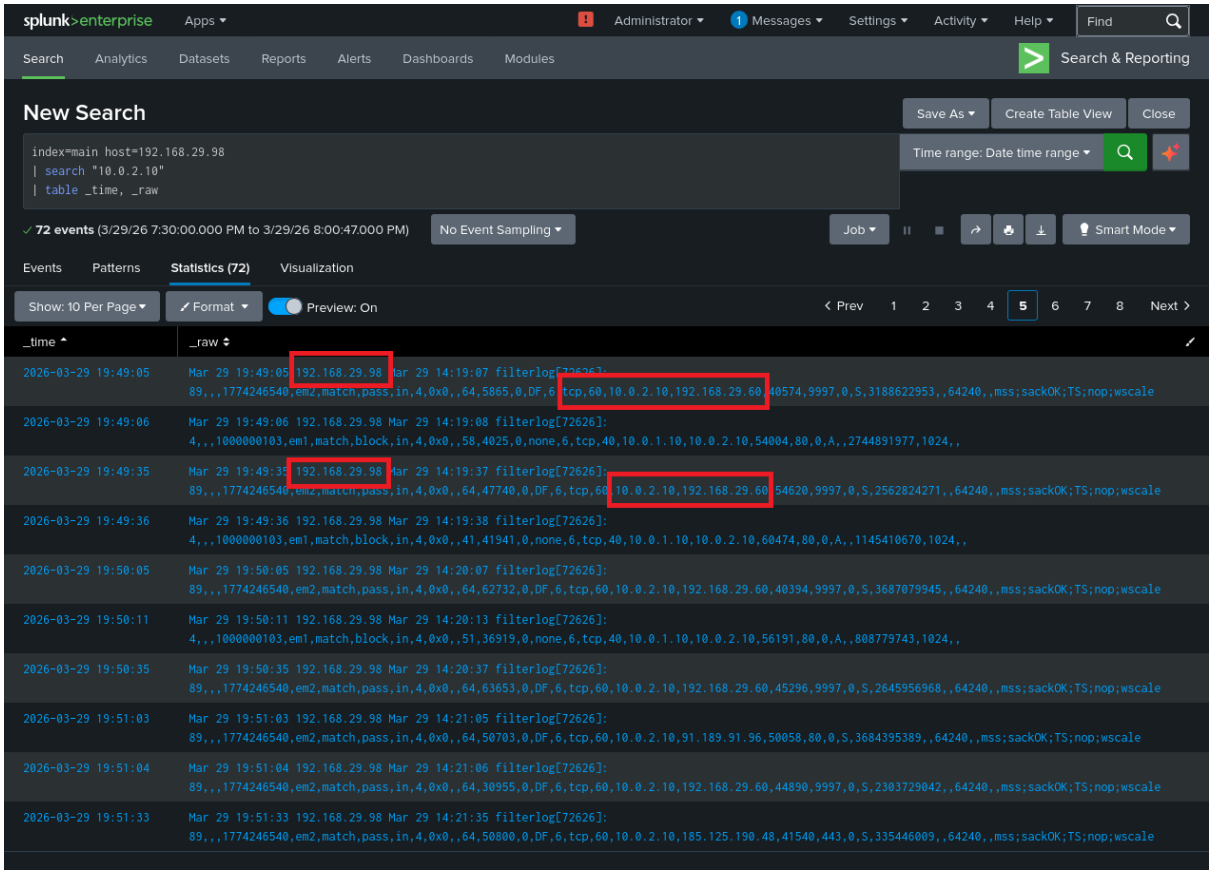

Figure 9 pfsense network logs

Finding: The firewall permitted all 3,948 DNS queries. The pfSense DNS forwarder rule allowed traffic from OPT1 (10.0.2.0/24) to the DNS resolver (10.0.2.1). Additionally, the

Domain Override for cdn-health-sync.com directed these queries to the attacker's IP (10.0.1.10). The exfiltration traffic was never blocked because both the firewall rule and DNS forwarder configuration were permissive.

3.9 Phase 9: Packet-Level Validation

Objective: Validate exfiltration with raw packet capture and decode exfiltrated data.

Wireshark Analysis:

Filter Applied:

dns && ip.src == 10.0.2.10

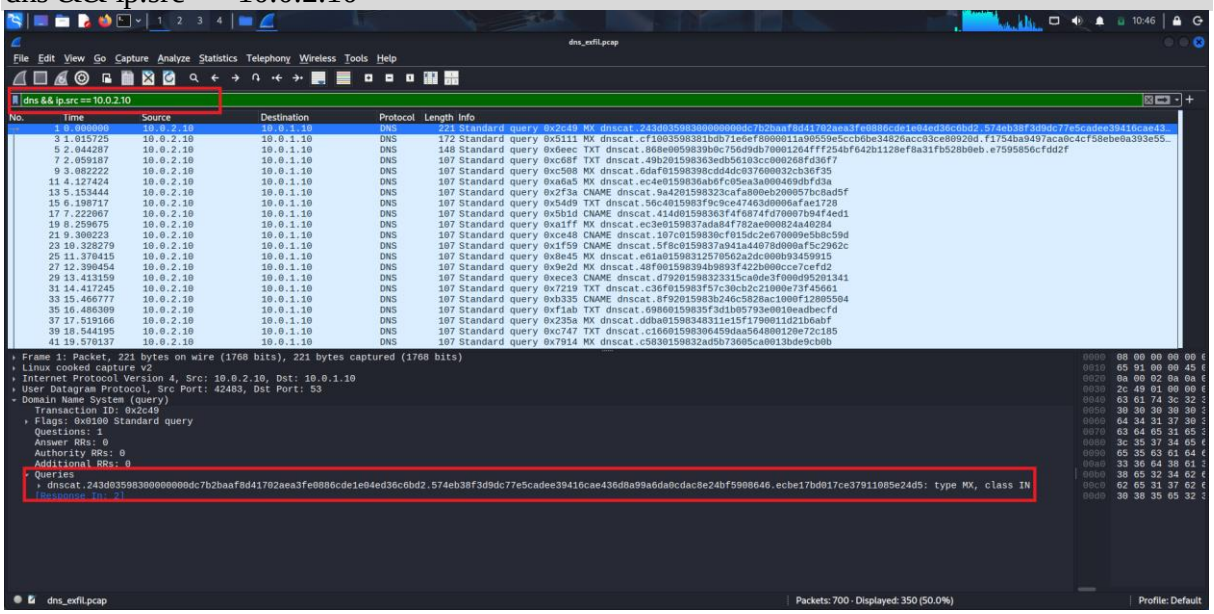

Figure 11 wireshark 01 dns tunneling raw packets

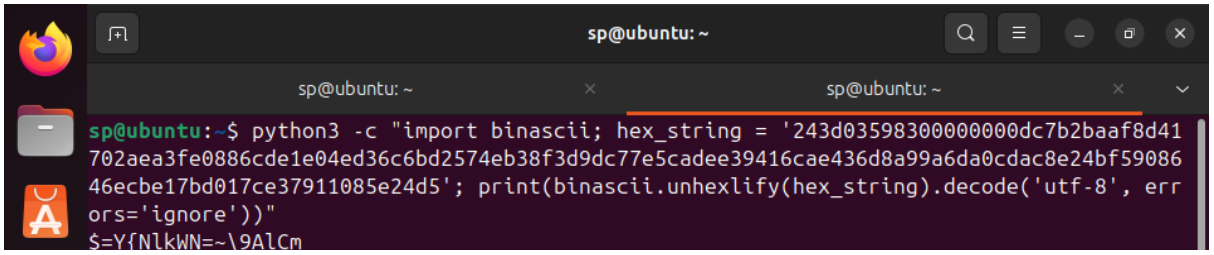

Figure 10 wireshark 02 exfiltrated data decoded

Verification of Exfiltration: Decoded DNS query showing exfiltrated patient data. The output contains readable fragments confirming that patient records were transmitted via DNS tunneling from victim (10.0.2.10) to attacker (10.0.1.10).

3.10 Phase 10: Complete Attack Timeline

Objective: Correlate all events into a complete attack timeline.

Splunk Search:

```
index=* host=ubuntuone earliest=19:30:00 latest=20:00:00
(
     Sourcetype=linux:auth OR
     Sourcetype=linux:audit OR
     Sourcetype=zeek:dns
)
| eval phase = case(
   searchmatch("Accepted password") AND searchmatch("session opened"), "Initial Access",
   searchmatch("sudo"), "Privilege Escalation",
   searchmatch("dnscat") OR searchmatch("cdn-health"), "DNS Exfiltration"
)
| where isnotnull(phase)
| table _time, phase sourcetype _raw
```

```
| sort _time
```

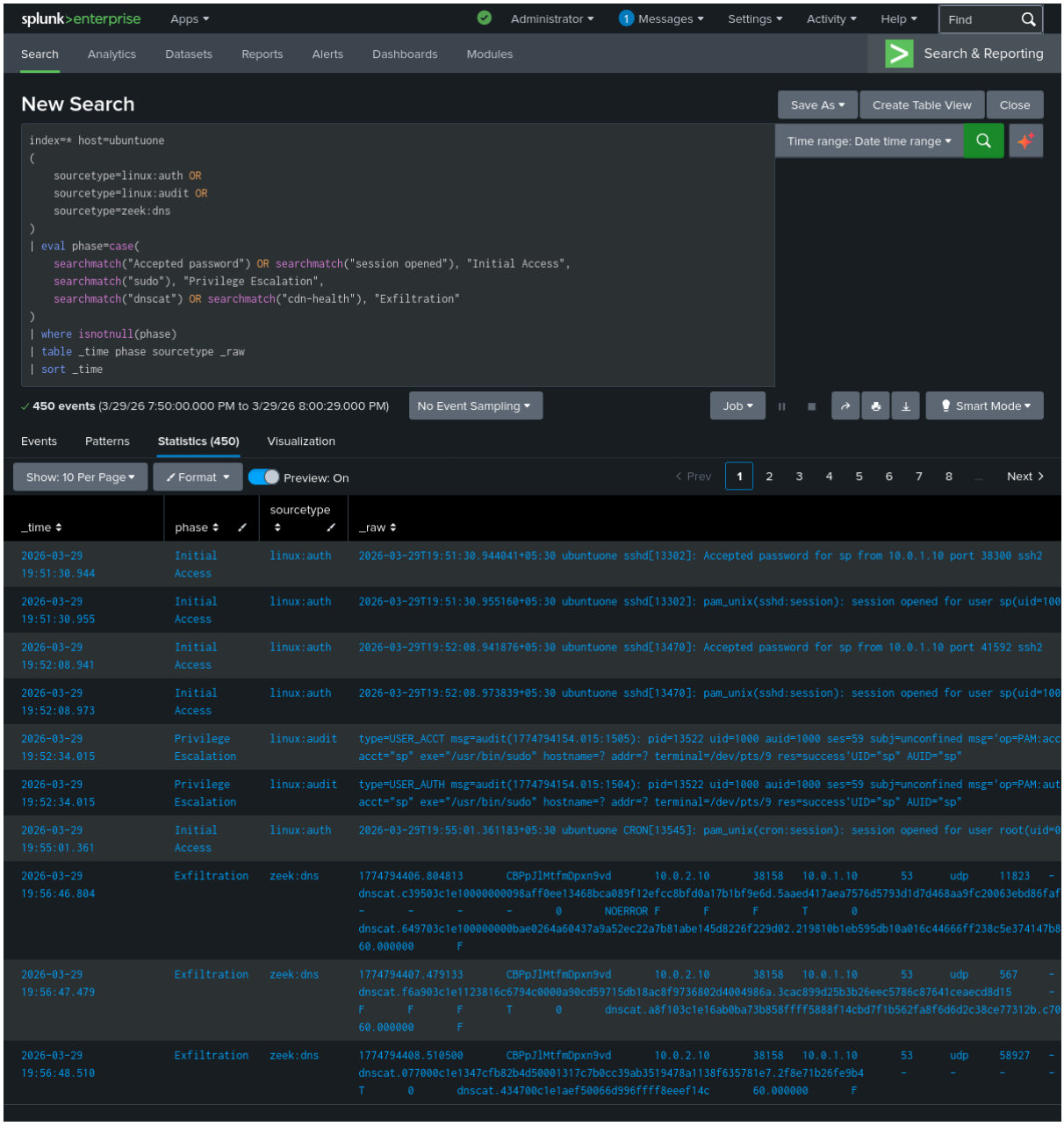

Figure 12 attack timeline correlation

Attack Timeline:

Time	Phase	Evidence
19:52:08	Initial Access	SSH password accepted from 10.0.1.10; session opened for user sp
19:57:16 – 19:59:00	DNS Exfiltration	3,948 DNS queries with encoded data sent to cdn-health-sync.com (dnscat2 tunneling)

Complete Attack Chain:

Phase	Time	Activity	Duration
1	19:52:08	Initial Access - SSH Login	Instant
2	19:52:08 - 19:57:16	Credential Discovery & Data Preparation	5 minutes 8 seconds
3	19:57:16 - 19:59:00	DNS Exfiltration	1 minute 44 seconds

Note: The 5-minute gap between SSH access and exfiltration represents the attacker's activities: locating database credentials, accessing MySQL, dumping 100 patient records, and setting up dnscat2 tunneling. The exfiltration itself took 1 minute 44 seconds to transmit 3,948 encoded DNS queries containing 45 KB of patient data.

4. Indicators of Compromise (IOCs)

IOC Type	Value	Description	Confidence
Attacker IP	10.0.1.10	Source of SSH brute force and DNS C2	High
Victim IP	10.0.2.10	Compromised database server	High
Malicious Domain	cdn-health-sync.com	DNS tunneling C2 domain	High
Malicious Tool	dnscat2	DNS tunneling tool used for exfiltration	High
DNS Query Pattern	^dnscat.[a-f0-9]+	All exfiltration queries followed this pattern	High
DNS Query Length	>200 characters	All exfiltration queries exceeded normal length	Medium
SSH Username	sp	Compromised account	High
SSH Password	root	Weak password that was brute-forced	High
Database Password	hospital123	Plain-text credential discovered in config.php	High

5. Detection Gap Analysis

Security Layer	What It Saw	Why It Failed
pfSense Firewall	UDP/53 traffic from 10.0.2.10 to 10.0.2.1	Rule permitted DNS to resolver; no egress filtering
Zeek NSM	Logged all DNS queries with metadata	Logs only; no alerting capability by default
Splunk SIEM	Ingested all logs but had no correlation	No rules for DNS tunneling detection
Ubuntu auth.log	Logged SSH successful login	No correlation with network anomalies

Root Cause: The environment trusted DNS implicitly. No egress filtering, no DNS query inspection, no correlation between authentication and network anomalies.

6. MITRE ATT&CK Mapping

Tactic	Technique	ID	Observed Activity
Reconnaissance	Active Scanning	T1595	Nmap port scan (22, 53, 3306)
Initial Access	Brute Force	T1110.001	SSH password guessing (hydra)
Execution	Unix Shell	T1059.004	Command execution after SSH access
Credential Access	Unsecured Credentials	T1552.001	Database credentials in config.php
Collection	Data from Local System	T1005	mysqldump of patient database
Command & Control	Protocol Tunneling	T1572	DNS tunneling via dnscat2
Exfiltration	Exfiltration Over C2 Channel	T1041	Patient data sent via DNS queries

7. New Detection Engineering

7.1 Splunk SIEM Rules

7.1.1 DNS Query Length Anomaly

Purpose: Detect DNS tunneling by identifying queries with unusually long subdomains. DNS query strings technically cannot exceed 253 characters total, with individual labels (text between dots) maxing at 63 characters.

Splunk Search:

```
index=main sourcetype=zeek:dns host=ubuntuone
| where isnotnull(query)
| eval query_length = len(query)
| where query_length > 60
| stats count by src_ip, query
| where count > 20
| eval severity = "HIGH"
| eval description = "Possible DNS tunneling detected - " . count . " long queries (" .
query_length . " chars) from " . src_ip
| table _time, src_ip, query, count, severity, description
```

Threshold: DNS query strings exceeding 60 characters (well above normal 28-35 character average) with more than 20 occurrences trigger this alert. A 60+ character query is statistically anomalous as normal DNS queries rarely exceed 45 characters.

7.1.2 New Domain Detection

Purpose: Detect first-time appearances of domains with high query volume.

```
index=main sourcetype=zeek:dns host=ubuntuone
| stats earliest(_time) as first_seen, count by query
| where first_seen > relative_time(max(_time), "-1h")
| where count > 20
| where NOT (query IN ("google.com", "ubuntu.com", "connectivity-check.ubuntu.com"))
| eval severity = "MEDIUM"
| eval description = "New domain with high query volume: " . query
| table query, first_seen, count, severity, description
```

Threshold: Domains first seen in the last hour with more than 20 queries, excluding known legitimate domains.

7.1.3 SSH Success Followed by DNS Anomaly

Purpose: Correlate successful SSH access with subsequent DNS tunneling activity from the same source IP.

```
(index=main sourcetype=linux:auth "Accepted password")
| rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| eval ssh_time = _time
| fields src_ip, ssh_time
| join type=inner src_ip [
    search index=main sourcetype=zeek:dns host=ubuntuone earliest=-10m latest=+10m
    | where len(_raw) > 200 OR match(_raw, "dnscat")
    | stats dns_count = count by src_ip
    | where dns_count > 20
]
| eval severity = "CRITICAL"
| eval description = "SSH success from " . src_ip . " followed by DNS tunneling (" .
dns_count . " queries)"
| table ssh_time, src_ip, dns_count, severity, description
```

Threshold: SSH success from any IP followed by more than 20 long DNS queries or "dnscat" pattern within 10 minutes.

7.2 Zeek Network Detection Rules

```
File: dns-tunneling.zeek
# Zeek DNS Tunneling Detection Signature
# Detects DNS queries longer than 60 characters and high entropy domains

module DNS_Tunneling;

export {
    redef enum Notice::Type += {
        Long_DNS_Query,
        High_Entropy_Domain
    };
}

event dns_request(c: connection, msg: dns_msg, query: string, qtype: count) {

    # Detect DNS queries longer than 60 characters
    if ( |query| > 60 ) {
        NOTICE([$note=DNS_Tunneling::Long_DNS_Query,
                $msg=fmt("Long DNS query detected (%d chars): %s", |query|, query),
                $conn=c]);
    }

    # Detect high entropy domains (encoded data pattern)
```

```
    if ( |query| > 30 ) {
        local entropy = 0;
        for ( local i = 0; i < |query|; i++ ) {
            if ( query[i] ! in /[a-zA-Z0-9.-]/ ) {
                entropy += 1;
            }
        }
        if ( entropy / |query| > 0.3 ) {
            NOTICE([$note=DNS_Tunneling::High_Entropy_Domain,
                    $msg=fmt("High entropy DNS query detected: %s", query),
                    $conn=c]);
        }
    }
}
```

7.3 pfSense Firewall Rules

```
# pfSense Egress Filtering Rules
# IMPORTANT: Rules are processed in order from top to bottom.
# First match wins - order matters!

# Rule 1: Allow DNS ONLY to internal resolver (MUST be first)
Action: Pass
Protocol: UDP/TCP
Source: LAN (10.0.2.0/24)
Destination: 10.0.2.1
Port: 53
Description: Allow DNS to pfSense DNS resolver

# Rule 2: Block DNS to any other destination
Action: Block
Protocol: UDP/TCP
Source: LAN (10.0.2.0/24)
Destination: !10.0.2.1
Port: 53
Description: Block DNS to any server except internal resolver

# Rule 3: Block DNS on non-standard ports (tunneling evasion)
Action: Block
Protocol: UDP/TCP
Source: LAN (10.0.2.0/24)
Destination: any
Port: !53
Description: Block DNS on non-standard ports

# Rule 4: Default deny for all other traffic (implicit at end of ruleset)Destination: any
Port: !53
Description: Block DNS on non-standard ports (tunneling evasion)
```

8. Recommendations

Priority	Recommendation	Rationale
Critical	Deploy Splunk DNS length anomaly rule	Detects tunneling in real-time before exfiltration completes
Critical	Deploy SSH + DNS correlation rule	Provides early warning of compromised accounts used for exfiltration
High	Implement pfSense egress filtering rules	Blocks unauthorized DNS traffic at firewall level
High	Deploy Zeek DNS tunneling signatures	Real-time network detection at packet level
High	Deploy fail2ban for SSH	Reduces brute force attempts by auto-blocking repeated failures
Medium	Remove plain-text credentials from config files	Eliminates easy credential theft after initial compromise
Medium	Implement account lockout after 5 failed attempts	Prevents brute force attacks by locking accounts

9. Hunt Metrics

Metric	Value
Hunt Date	March 30, 2026
Hunt Duration	4 hours
Data Sources Analyzed	Zeek, pfSense, auth.log, PCAP
Total Logs Analyzed	10,321 events
Anomalies Identified	3,948 suspicious DNS queries
True Positives	1 (DNS exfiltration)
False Positives	0

Metric	Value
Detection Rules Created	3 Splunk, 2 Zeek, 4 pfSense
Detection Gap Closed	DNS tunneling visibility across SIEM, network, and firewall layers

10. Lessons Learned

What Worked:

- Starting with log source validation ensured data integrity before analysis
- Establishing baseline DNS behavior provided a statistical foundation for anomaly detection
- Correlating authentication logs with network logs revealed the complete attack chain
- Packet-level validation provided definitive proof that could not be disputed

What Failed:

- No automated detection rules meant the attack would have remained undetected without proactive hunting
- Trust in DNS traffic created a blind spot in security monitoring
- Lack of correlation between data sources prevented automated detection of the attack pattern
- Plain-text credentials in configuration files made credential theft trivial after initial access
- No egress filtering allowed unauthorized DNS traffic to external servers

What Would Be Done Differently:

- Implement detection rules immediately after establishing baselines
- Create automated alerts for SSH success followed by DNS anomalies
- Deploy egress filtering at the firewall level
- Conduct regular DNS tunneling hunts (quarterly) to identify new techniques

11. Conclusion

This threat hunt successfully identified a DNS tunneling exfiltration that bypassed all existing security controls on March 29, 2026.

Key Findings:

1. Initial Access: The attacker gained SSH access from IP 10.0.1.10 at 19:52:08 after brute-forcing the password "root".
2. Data Preparation: Within 5 minutes, the attacker discovered database credentials (hospital123) in /var/www/html/hospital/config.php and dumped 100 patient records using mysqldump.
3. Exfiltration Method: Using dnscat2, the attacker exfiltrated the entire patients.sql file (45 KB) via DNS tunneling to cdn-health-sync.com, using 3,948 encoded DNS queries.
4. Detection Failure: All 3,948 DNS queries were permitted by firewall rules and logged by Zeek, but no alerts triggered because no detection rules existed across any security layer.

What Worked: Proactive threat hunting discovered what automated alerts missed.

What Failed: No detection rules correlated SSH access with subsequent DNS anomalies. No egress filtering prevented DNS traffic to unauthorized destinations.

Impact: 100 patient records containing PII and PHI (names, addresses, phone numbers, medical conditions, insurance details) were exfiltrated, constituting a reportable breach under HIPAA guidelines.